

2026-1 DataMining

데이터마이닝 중간발표

01. Data Collection & Management

02. Viewer-Chat Mismatch EDA

03. Modeling



산업공학과 조시훈, 산업공학과 권성안,
산업공학과 LE

CONTENTS

01 Data Collection & Management

- 표본 설계: Top 100 Pool → Random 15
- 실시간 수집: Viewer Snapshot + WebSocket Chat
- 데이터 관리: Raw Data 저장과 Minute Feature 생성

02 Viewer-Chat Mismatch EDA

- 데이터 결측치 초기처리
- 데이터 품질과 chat QC Bucket
- Distribution and Time Structure
- Viewer-Chat 관계와 Time-lag 반응
- Clustering

03 Modeling

- Isolation Forest
- Decision Tree
- Naïve Bayes
- Classifying Neural Network

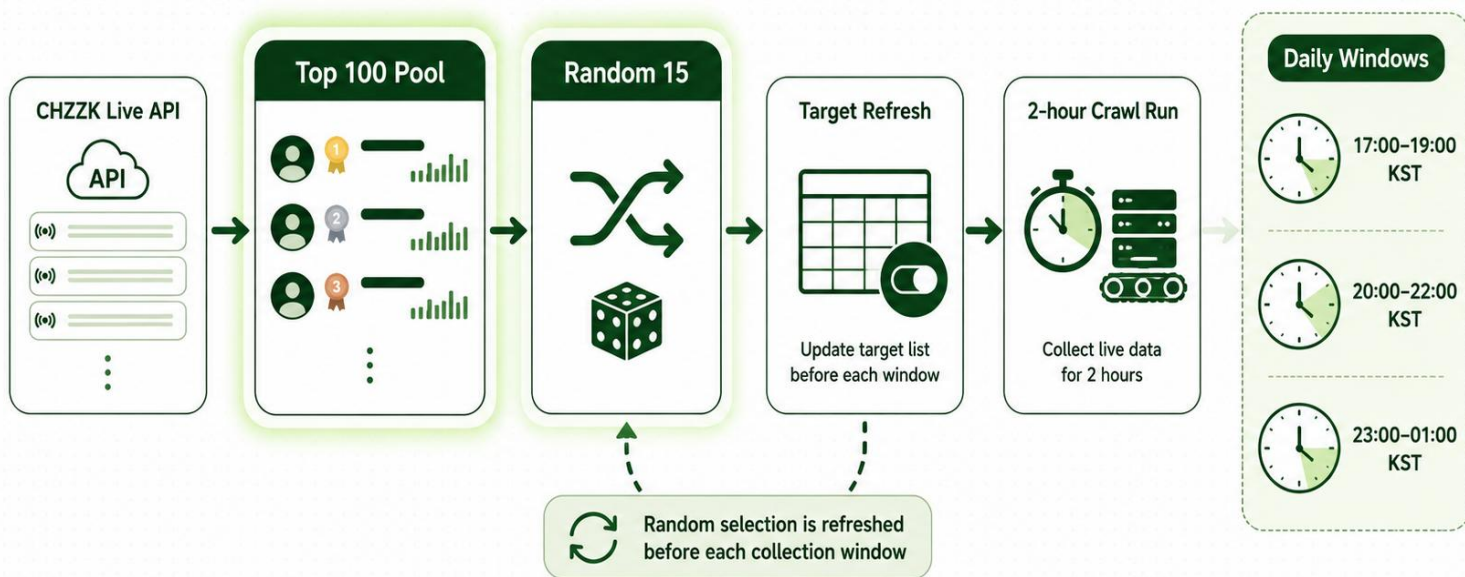
PART. 01

Data Collection and Management

1.1. Sampling Design: Top 100 Pool -> Random 15

매 수집 window마다 상위 라이브 풀을 다시 만들고 그중 15개 채널을 무작위 추출

Livestream Crawler Data Sampling Pipeline



From broad discovery to focused, time-bound collection.

Random. Refreshed. Repeatable.

Data 매 수집 window마다 CHZZK 현재 라이브 목록의 상위 100개 방송 pool을 만들고 그 중 15개 채널을 무작위 선택해 수집 target을 갱신

build_pool.py: CHZZK /open/v1/lives에서 현재 live pool 수집
-> viewer 기준 top100 구성
random.sample: Top 100 pool에서 Random 15개 선택 -> top15_targets.csv 저장
setup.py: 기존 active target 비활성화 후 새 15개 target만 활성화
cron: 17-19시, 20-22시, 23-01시 KST에 각 2시간 자동 실행

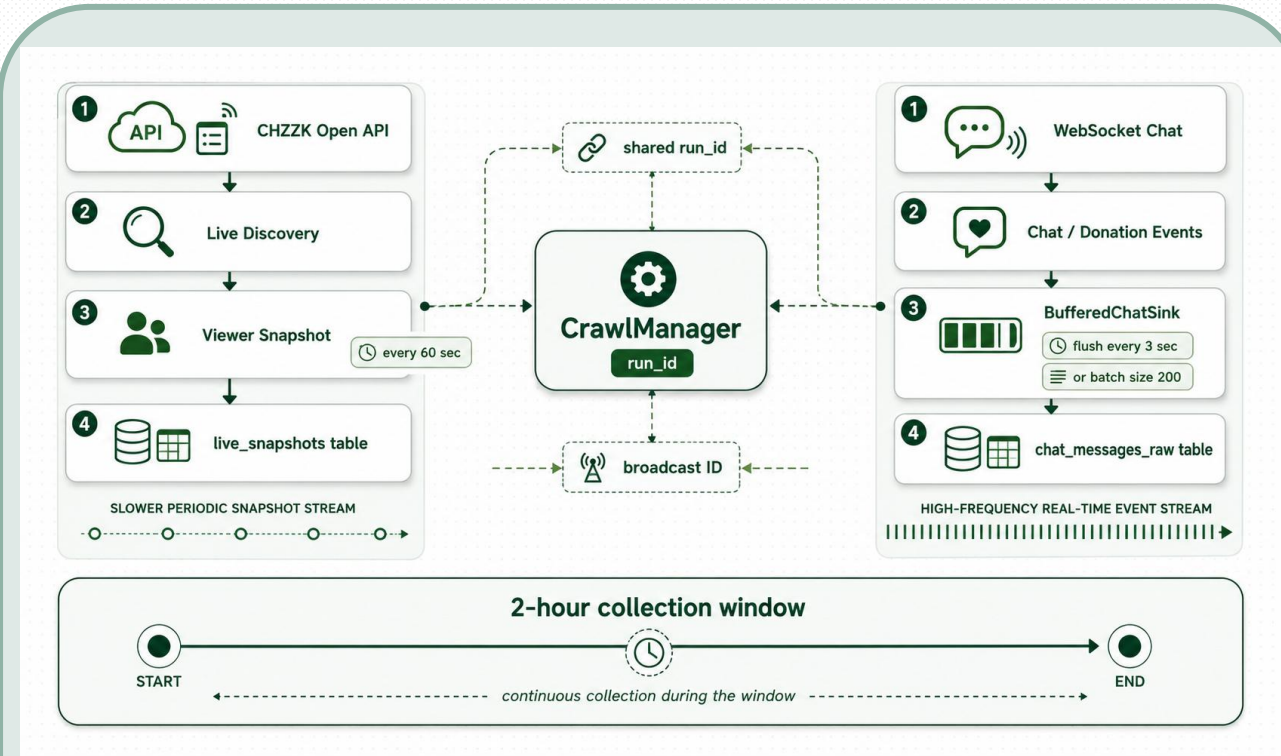
Data수집 대상을 고정하지 않고 현재 라이브 방송 중 viewer 수 기준 상위 약 100개를 pool로 만들고 그 안에서 15개를 무작위로 뽑는다.

그 후 기존 active target을 모두 끄고 새로 뽑힌 15개만 활성화.

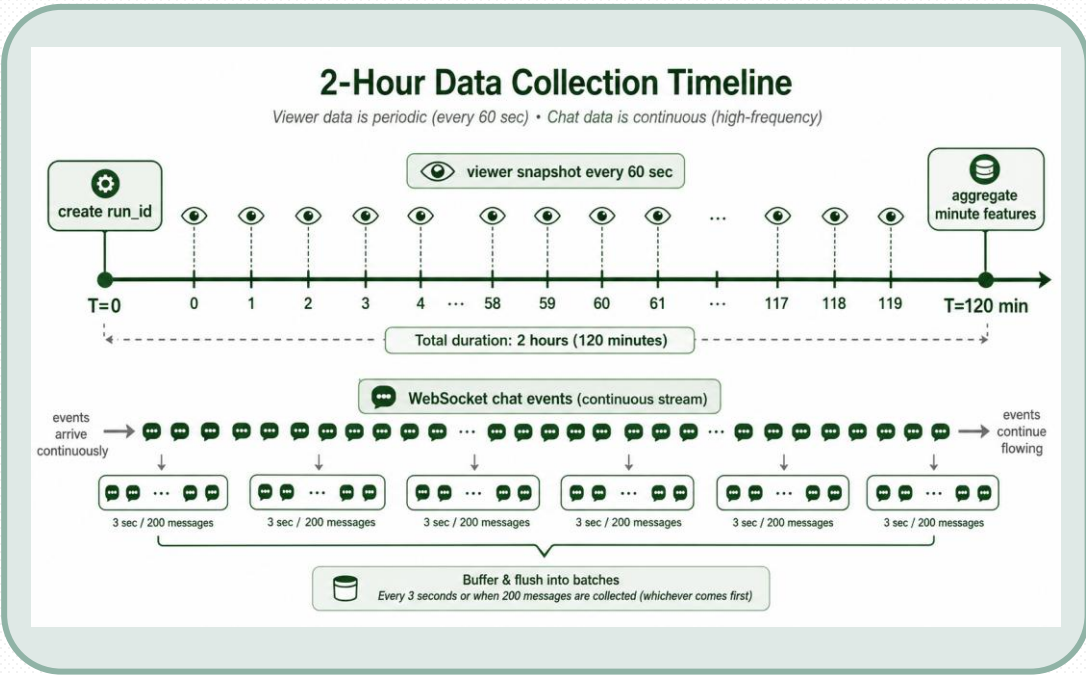
따라서 매 window마다 다른 방송이 들어올 수 있고 고정 스트리머만 반복 수집하는 편향을 줄이도록 설계

1.2. 실시간 수집: Viewer Snapshot & Websocket Chat

하나의 run_id 안에서 viewer snapshot과 WebSocket chat event를 동시에 수집



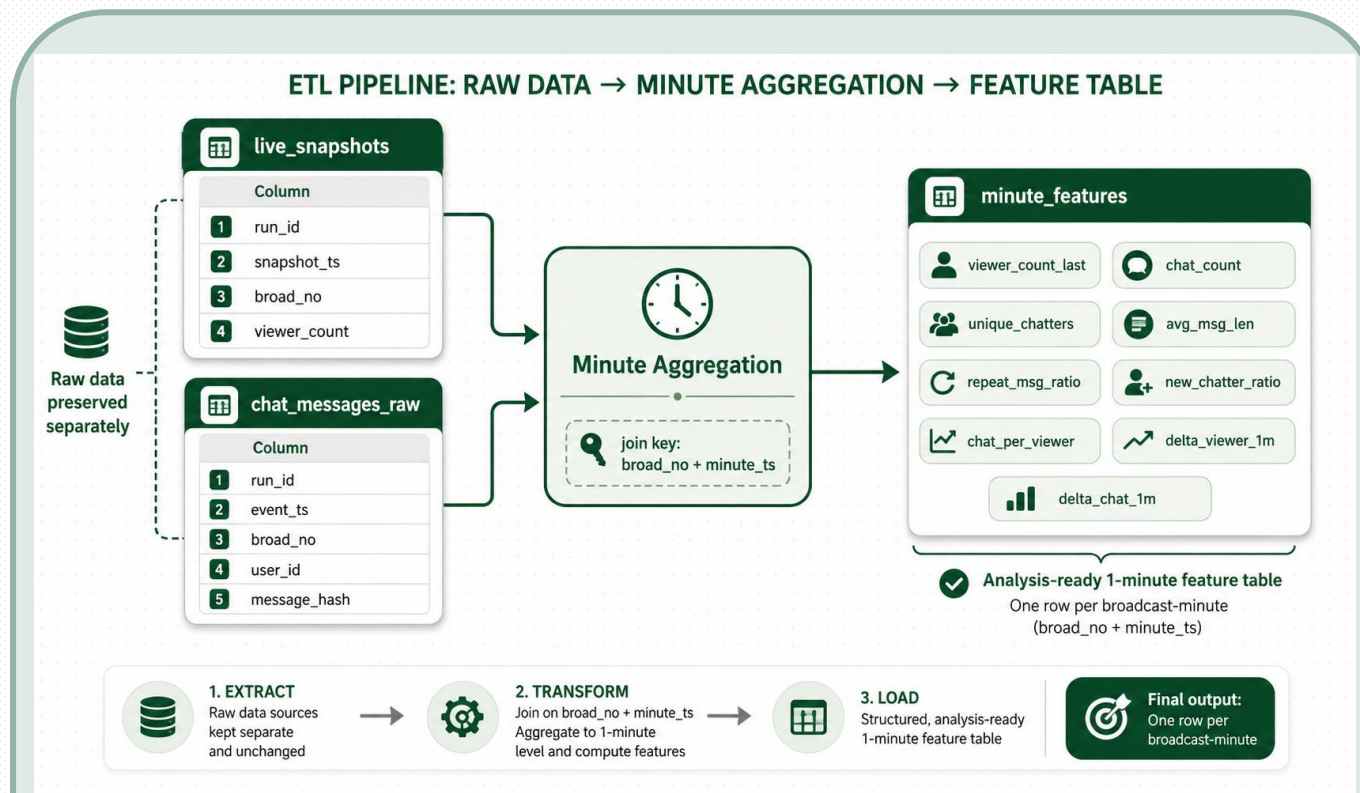
Run control: CrawlManager가 run_id를 생성하고 2시간 collection window를 관리
Viewer stream: active target의 live 상태를 조회해 60초마다 viewer snapshot 저장
Chat stream: 각 broad_no별 WebSocket collector가 chat/donation event 수집
Buffering: chat event는 3초 또는 200건 기준으로 batch insert



역할	수집 window 실행	전체 수집 제어 /run_id 생성	수집 주기 설정	Viewer live 조회	WebSoc ket chat 수집	Chat buffer/ batch insert	저장 테이블 구조
코드 파일 명	run_pilot.py	manager.py	settings.py	live_discovery.py	chat_wss.py	base.py	models.py

1.3. 데이터 관리: Raw Data 저장과 Minute Feature 생성

Raw snapshot과 raw chat event를 보존한 뒤, broad_no+minute_ts 기준으로 1분 단위 분석 테이블을 생성



Raw table → Minute feature

live_snapshots와 chat_message_raw를 raw_table로 보존하고, event_ts와 snapshot_ts를 분 단위 minute_ts로 맞추고 broad_no + minute_ts 기준으로 minute_feature를 생성

1. Timestamp alignment

- chat event_ts → minute_ts
- snapshot_ts → minute_ts

2. Chat aggregation

- broad_no + minute_ts 기준 groupby
- chat_count, unique_chatters, avg_msg_len 생성

3. Viewer aggregation

- 같은 minute 안의 마지막 viewer_cnt를 viewer_count_last로 사용

4. Merge & feature generation

- Viewer snapshot과 chat 집계를 broad_no + minute_ts 기준으로 결합
- chat_per_viewer, delta_viewer_1m, delta_chat_1m 생성

Stream alignment

- chat이 없는 minute은 chat_count = 0, unique_chatters = 0으로 보존
- viewer snapshot은 60초 단위이므로 같은 broad_no 안에서 최대 1분 forward-fill
- viewer_count가 0 이하이면 chat_per_viewer = 0으로 처리

Run validation

- verify_run.py: snapshots / chats / features row count 확인
- export_csv.py: minute_features와 raw chat을 CSV·Excel로 반출

PART. 02

Viewer Chat mismatch EDA

2.0. 데이터 결측치 초기 처리

```
loaded_file_cnt  row_cnt  column_cnt
0                20      20519      18
```

```
29 -> 13일 오후 11시 27분 ~ 오전 1시 27분
30 -> 14일 오전 2시 ~ 2시 30분
31 -> 14일 오후 5시 ~ 7시 1분
32 -> 14일 오후 8시 ~ 10시 1분
33 -> 14일 오후 11시 ~ 오전 1시
34 -> 15일 오후 5시 ~ 7시
35 -> 15일 오후 8시 ~ 10시
36 -> 15일 오후 11시 ~ 오전 1시
37 -> 16일 오후 5시 ~ 7시 1분
38 -> 16일 오후 8시 ~ 10시
39 -> 16일 오후 11시 ~ 오전 1시
40 -> 17일 오후 5시 ~ 7시
41 -> 17일 오후 8시 ~ 10시
42 -> 17일 오후 11시 ~ 오전 1시
43 -> 18일 오후 5시 ~ 7시
44 -> 18일 오후 8시 ~ 10시
45 -> 18일 오후 11시 ~ 오전 1시
46 -> 19일 오후 5시 ~ 7시
47 -> 19일 오후 8시 ~ 10시
48 -> 19일 오후 11시 ~ 오전 1시
```

Data Reading

초기 데이터는 총 20개의 파일로 구성되어 있었다. 초기에 설계상 7일간 매일 (17~19시), (20~22시), (23~01시)에 수집하기로 했기에 EC2 server의 불안정성 등의 원인으로 잘못된 시간에 수집된 29,30번 파일은 배제했다.

```
data (행,열): (19116, 18)
=====전체 결측치 관찰=====
feature_id      0
broad_no        0
minute_ts       0
user_id         0
category_id     0
viewer_count_last 4
chat_count      0
unique_chatters 0
avg_msg_len     1736
repeat_msg_ratio 0
new_chatter_ratio 0
chat_per_viewer 0
delta_viewer_1m 189
delta_chat_1m   184
run_id          0
created_at      0
updated_at      0
source_file     0
dtype: int64

총 처리해야 할 결측치는 2113
viewer가 0인 것: 0
chat이 0인 것: 1736
결정된 결측 비율 : 0.02%
위치 33 위치 34 위치 38 위치 48
```

Missing Value

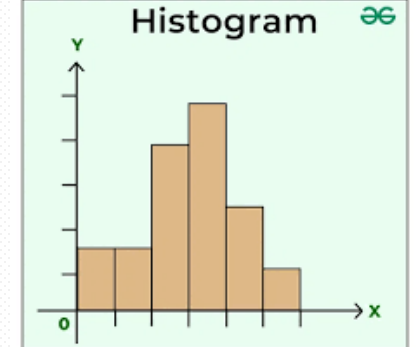
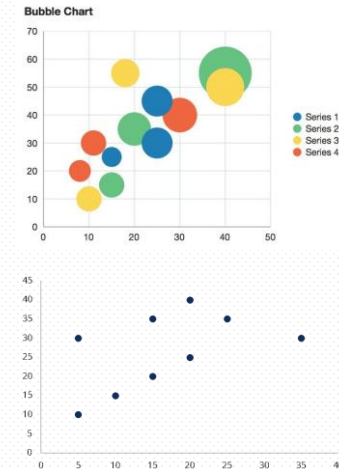
명시적인 결측치는 해당 분이 종료될 때 측정된 시청자수인 viewer_count_last 4개의 row였다. 이는 전체 데이터의 0.02%에 해당되는 양으로 매우 적어 선행보간을 통해 채워넣었다. Avg_msg_len은 0chat으로 인한 결측이라 판단하고 0chat의 경우 avg_msg_len을 0으로 지정해 넣어줬더니 우측과 같이 결측이 사라졌다.

```
feature_id      0
broad_no        0
minute_ts       0
user_id         0
category_id     0
viewer_count_last 0
chat_count      0
unique_chatters 0
avg_msg_len     0
repeat_msg_ratio 0
new_chatter_ratio 0
chat_per_viewer 0
delta_viewer_1m 184
delta_chat_1m   184
run_id          0
created_at      0
updated_at      0
source_file     0
```

```
=== 전체 0-chat 방송 세션 분리 결과 ===
전체 row(df_all): 19116
행동분석 row(df_behavior): 18457
QC bucket row(df_zero_bucket): 659
전체 0-chat 방송 세션 수: 8
```

EDA용 Data

본 한 Session 묶음 전체에 대해 viewer_count_last가 0인 Session은 일반적인 viewer-chat 행동관계 분석을 왜곡할 수 있기 때문에 EDA 상에서는 이를 배제하고 진행하고 모델링과정에서는 따로 QC_bucket을 만들어두어 배제하지 않고 진행하려 했다.



2.1. 데이터 품질과 0-Chat QC-Bucket

오랜 기간 지속된 0chat을 이상치라고 볼 수 있는가?

item	value	note
전체 broadcast session	184	전체 0-chat 포함
전체 0-chat session	8	행동분석 제외, QC bucket 보존
전체 0-chat minutes	659	QC bucket rows
행동분석 session	176	전체 0-chat 제외
10분 이상 session	173	clustering/anomaly 분석 기준
user_id+run_id 중 복수 broad_no	4	최종 EDA는 run_id+broad_no 기준

Session Definition

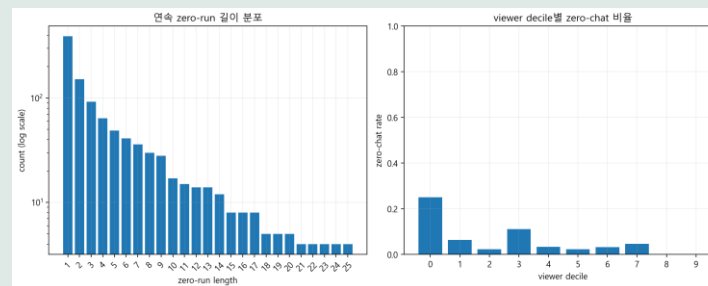
본 EDA의 session은 run_id + broad_no이다. 즉, 한 수집 window 안에서 관측된 하나의 실제 방송을 분석 단위로 둔다.

Why not user_id + run_id?

User_id + run_id는 한 스트리머가 한 수집 window에 들어온 기록이다. 하지만 표에서 볼 수 있다 싶이 같은 run 안에서 방송을 재시작한 Session이 4개 발견되었기 때문에 실제 방송 단위인 run_id+broad_no로 수행했다.

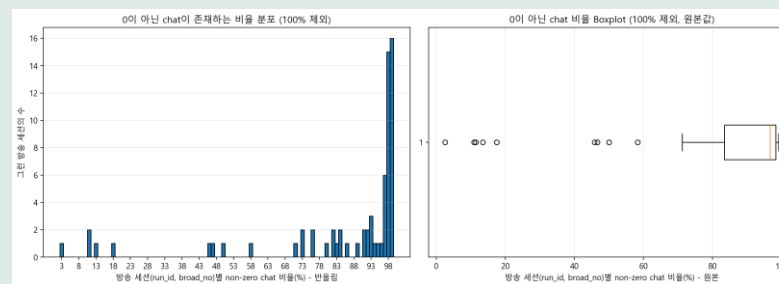
0-chat Handling

전체 0-chat session은 EDA에서는 제외했지만 삭제하지는 않았다. WebSocket 미수집 가능성 또는 극단적 viewer-chat mismatch 후보일 수 있으므로 QC-Bucket으로 보존했다.



zero-run plot: 행동분석 session 안에서 짧은 0-chat 구간은 흔하지만, 긴 zero-run은 드물다.

viewer decile plot: zero-chat이 특정 시청자 구간에서 일반적으로 많이 발생하기보다, 일부 session-level 특성으로 따로 관리해야 함을 시사.



All-zero Session을 제외한 뒤에도 대부분의 방송 세션은 거의 모든 minute에서 chat이 관측됨. 일부 boxplot의 왼쪽 outlier들은 non-zero chat 비율이 낮다. 이들은 완전 0-chat은 아니지만, 긴 침묵 구간이나 sparse chat이 존재할 수 있다. 그래서 All-zero Session은 QC bucket으로 분리하고, 남은 behavior set 안에서는 zero-run length와 zero-chat rate를 추가 feature로 본다.

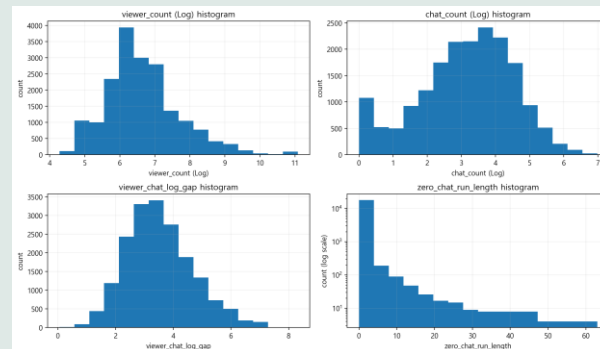
2.2. Distribution & Time Structure

Heavy tail 분포와 session trajectory

metric	median	Q1	q3
median_viewer	633	412	1271
mean_chat_per_min	30.890	12.267	64.273
median_gap	3.360	2.796	4.042
zero_chat_rate	0.000	0.000	0.020

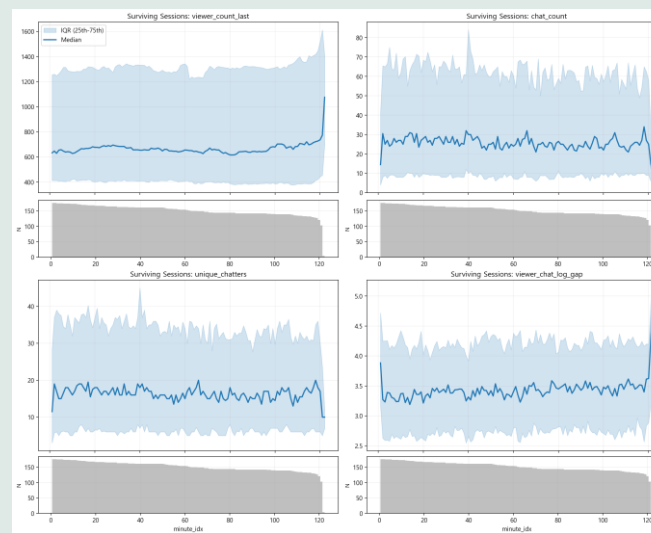
How to read this table?

- **median viewer**
각 broadcast session 안에서 viewer_count_last의 중앙값을 먼저 계산, 그 session-level 값들의 중앙값을 표시한 값.
- **mean chat_per_min**
각 session 안에서 1분당 chat_count 평균을 계산, 그 session-level 값들의 중앙값을 표시한 값.
- **median gap**
minute별 $\log_{10}(\text{viewer}) - \log_{10}(\text{chat})$ 을 계산, session 내부 중앙값을 구하고, 그 session-level 값들의 중앙값을 표시한 값.
- **zero_chat_rate**
각 session 안에서 chat_count = 0인 minute 비율. 중앙값이 0이라는 것은 대부분의 session에서 0-chat minute이 없다는 뜻.



viewer-chat gap histogram: 대부분의 minute row는 gap이 2~5 근처에 몰려 있지만, 오른쪽 꼬리가 있어 viewer 대비 chat이 약한 구간이 존재한다.

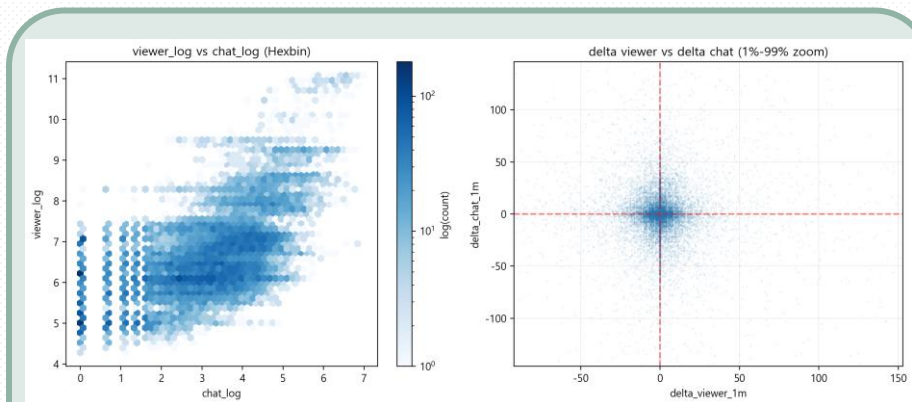
zero-run length histogram: 0-chat 구간은 대부분 짧지만, 일부 session에서는 긴 zero-run이 나타남. 단순히 chat=0여부보다 연속 지속시간을 feature로 보는 것이 필요



- 각 line은 minute_idx별 session media이고 음영처리된 구간은 IQR이다. 아래 회색 bar은 해당 minute_idx까지 남아있는 surviving session 수이다.
- viewer, chat, unique는 전체적으로 큰 급변 없이 유지되지만 IQR이 넓어 session간 차이가 크다.
- 후반부 viewer_chat_log_gap의 급등은 이상현상이 아니라 남아 있는 session수가 줄어들어 따라 발생했을 수 있기에 단정하지 않는다.

2.3. Viewer-Chat Coupling & Time Lag

Scale level correlation vs minute-level response



- viewer_log vs chat_log(Hexbin)

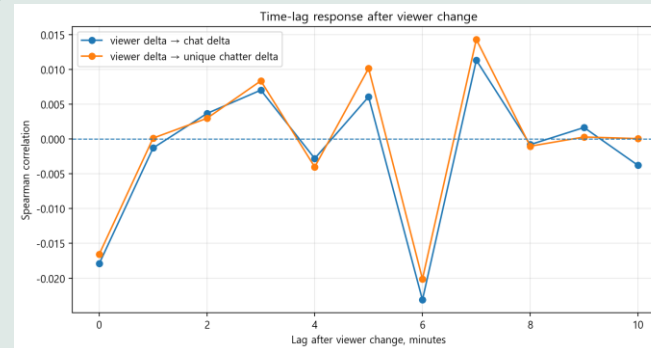
규모 관계와 minute level coupling을 한 번에 비교하게 해준다. Viewer가 큰 minute 일수록 chat도 많아지는 경향을 보인다. 즉 방송 규모 차원에서는 viewer와 chat 사이에 양의 상관관계를 땀을 알 수 있다.

- delta viewer vs delta chat

- 점들이 원점 주위에 밀집해 있다. 이는 viewer가 1분 사이 증가했다고 해서 chat도 같은 minute에 같이 증가한다고 보기 어려움을 의미한다.

Level Relation은 존재하지만 Dynamic Response는 약하다!

Viewer-chat mismatch는 단순 delta correlation보다 session-level gap과 zero-chat persistence로 보는 것이 적절!



Time-lag response check: viewer 변화가 발생한 minute t 를 기준으로 chat 변화와 unique chatter의 변화가 $t+0$ 부터 $t+10$ 까지 따라오는지를 spearman correlation으로 계산한 것이다.

-> 모든 lag에서 correlation이 매우 작다. 전체적으로 값이 -0.02에서 0.015근처에 머물고 특정 lag에서 일관되게 높아지는 패턴도 없다.

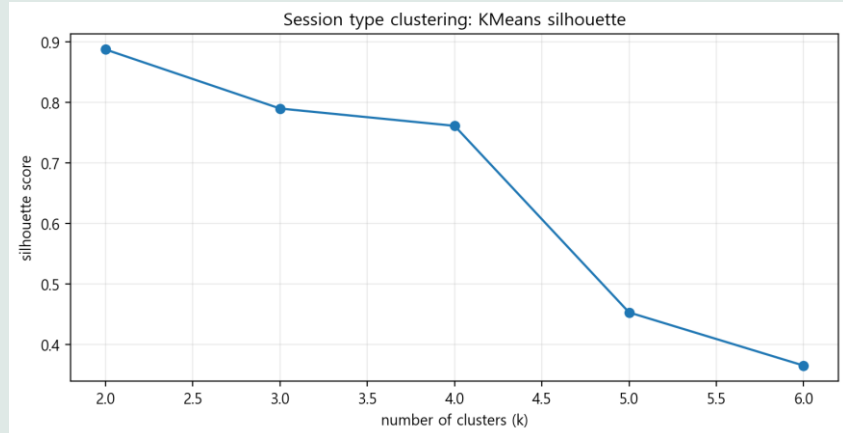
즉시 반응도 약하고 0~10분 지연 반응도 약하다!

pair	pearson	spearman
viewer_log vs chat_log	0.588	0.570
viewer_log vs unique_log	0.636	0.618
delta_viewer_1m vs delta_chat_1m	-0.024	-0.018
chat_log vs viewer_chat_log_gap	-0.683	-0.643

- viewer_log vs chat_log 와 viewer_log vs unique_log는 모두 양의 상관을 띤다. 이는 방송 규모가 커질수록 채팅 수와 고유 채팅자 수도 함께 증가함을 의미한다. Viewer와 chat 사이의 규모 수준 관계는 존재한다.
- delta_viewer vs delta_chat은 0에 가깝다. 이는 viewer가 1분 동안 변화하더라도 chat이 같은 방향으로 선형적으로 반응한다고 보기는 어렵다는 의미이다.
- chat_log vs viewer_chat_log_gap은 음의 상관관계가 크다. Gap의 정의에 의해 chat이 적을수록 gap이 커지는 것은 자연스럽다. Viewer 대비 chat 반응이 약한 구간을 잡기 위한 보조지표로 연산해 두었다.

2.4. Clustering

Kmeans Clustering

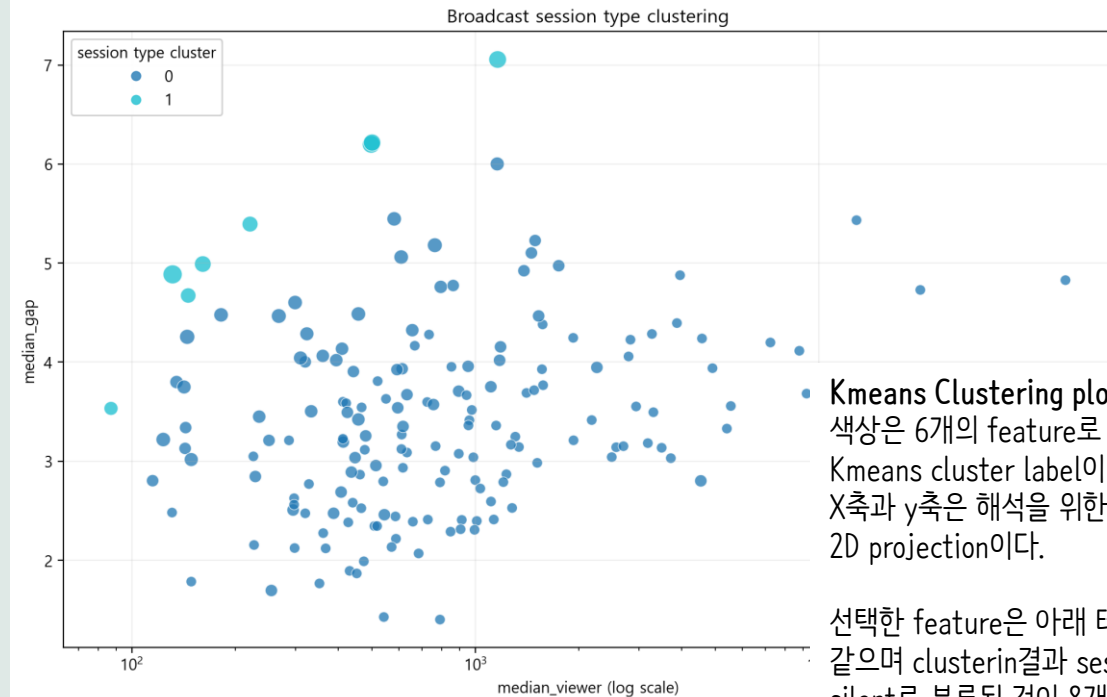


적절한 cluster 수 선정

K 값이 증가할 수록 silhouette score가 감소한다. 따라서 가장 높은 score 0.887을 주는 2개로 분류하는 것이 적절하다.

Cluster의 의미

클러스터링 된 결과를 보면, 다른 데이터 대비 비정상적으로 높은 median-gap을 가진 8개의 세션이 다른 군집으로 분류되어 있다. 따라서 이를 세부 군집이 아니라 active vs silent split로 해석했다. 즉, 1로 분류된 군집 자체가 어느 정도 높은 확률로 chat-bot 의심 세션으로 분류할 수도 있을 것이다.



Kmeans Clustering plotting
색상은 6개의 feature로 수행한 Kmeans cluster label이다.
X축과 y축은 해석을 위한 2D projection이다.

선택한 feature은 아래 테이블과 같으며 clusterin결과 session_type silent로 분류된 것이 8개, active로 분류된 것이 165개이다.

session_type_cluster	n_sessions	median_viewer	mean_chat_per_min	mean_unique_per_min	zero_chat_rate	median_gap	max_zero_run	session_type
1	8	191	0.838	0.542	0.742	5.192	17	sparse/silent
0	165	658.5	53.996	28.821	0.024	3.269	0	normal/active

PART. 03

Baseline Models

3.1. Isolation forest 기반

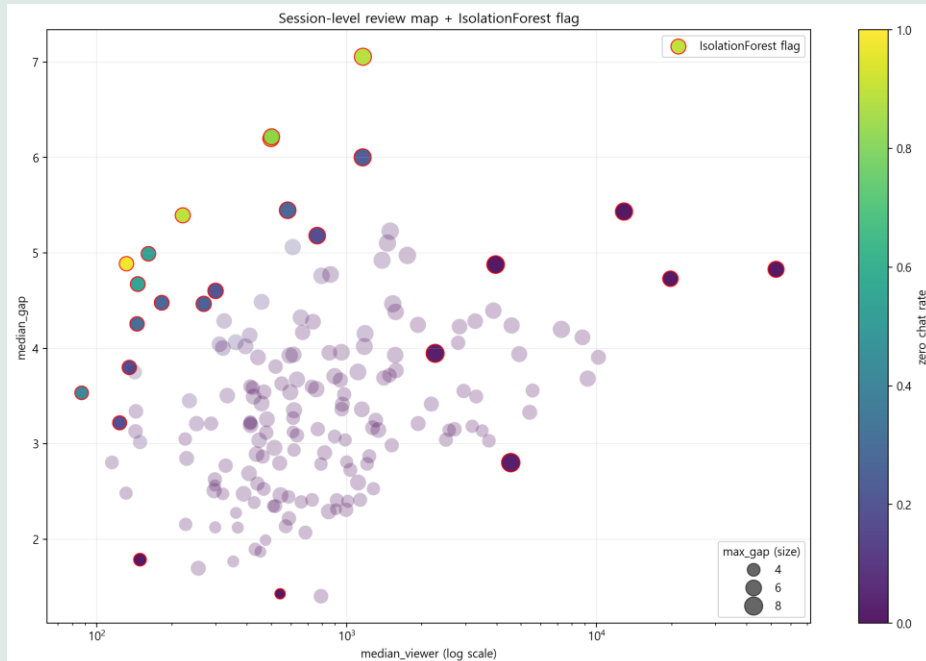
Isolation forest기반 고립 세션 flagging

Why Isolation Forest First?

iForest는 기본 모델로 채택하기에 상당히 복잡하다.

하지만 근본 자체가 이상치 탐색 모델로, 다른 데이터들과 이질적인 데이터들을 뽑아내는 모델이다. View-Bot을 쓰는 Streaming의 수가 소수일 것이라 판단, Isolation Forest Model로 이를 식별할 수 있을 것이라 판단했다.

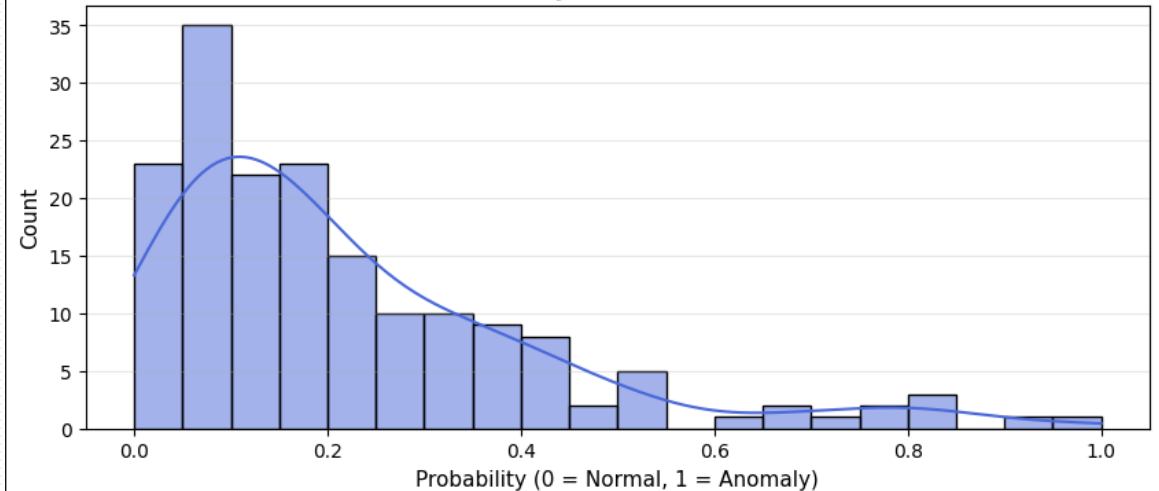
모델 진행 결과



10분 이상 broadcast session 173개중 25개 review candidate 시각화

- **X축: median_viewer**
각 broadcast session 안에서 viewer_count_last의 중앙값으로 log scale이다
- **Y축: median_gap**
각 minute의 viewer, chat log gap을 계산한 뒤 session 안에서 중앙값
- **색: zero_chat_rate**
해당 session 안에서 chat_count = 0인 minute 비율
- **점 크기: max_gap**
Session 안에서 가장 컸던 viewer_chat gap
- **빨간 테두리: IsolationForest가 feature상으로 고립되게 분류한 세션**
- **라벨: IsolationForest score 상위 세션**

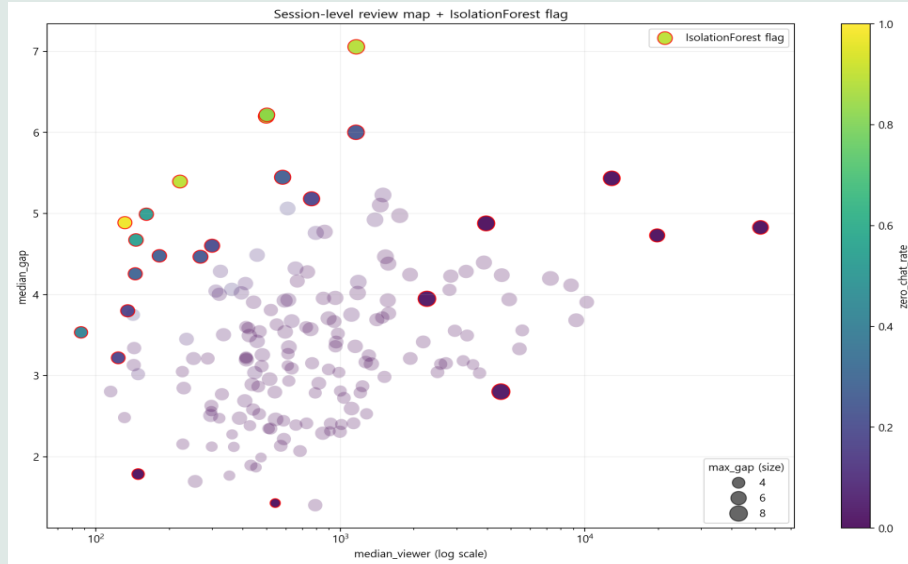
Viewbot Probability Distribution (All Sessions)



3.1. Isolation forest 기반

Isolation forest기반 고립 세션 flagging

모델 진행 결과



결과, View-Bot 의심이 타당한 데이터들이 다수 포함되어 있음.

- 특히 많은 데이터들이 median-gap 수치가 높음.

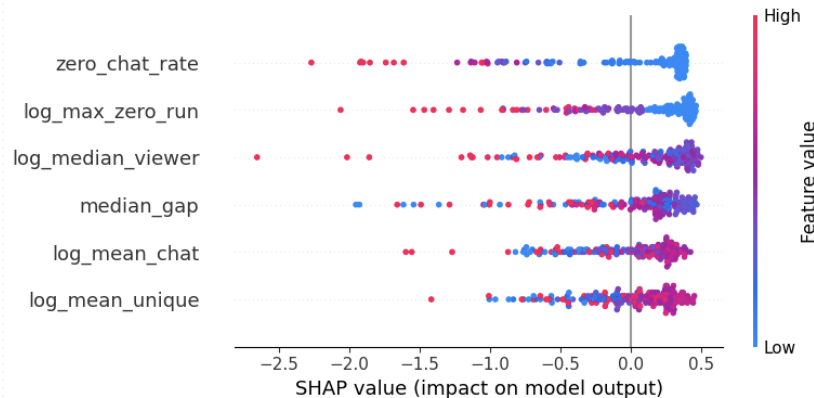
이는 View-Bot을 사용중이라면 시청자 수가 채팅 수보다 월등히 높을 것이라는 상식적 판단과 일치

- 이는 더 나아가 View-Bot 의심 사용 스트리밍이 다양한 차원에서 일반적인 스트리밍과 구분됨을 뒷받침하기도 함.

단순히 median-gap 수치만 차이가 있었다면, anomaly로 판단되기에는 부족할 것임. Anomaly로 판별되었다는 것은 다른 feature 역시 일반적 steaming들과 차이를 보인다는 의미로, 이는 다양한 차원에서 View-Bot이 Normal Streaming과 차이점을 가진다는 것을 내포함.

실제로, 진행된 SHAP 분석 결과를 보면, median-gap 수치 뿐만 아니라 zero_chat_rate 등 다양한 feature 역시 anomaly detection에 적극적으로 사용된 것을 볼 수 있음.

이는 간단한 Logistic Regression을 넘어 Neural Network이나 AutoEncoder 등의 복잡한 신경망 모델을 바탕으로 학습을 진행할 필요성을 역설함. 수학적 표현력이 뛰어난 모델은 View-Bot Steaming이 보이는 미묘한 차이를 잡아낼 수 있을 것이다.



- 하지만 다음과 같은 한계점도 발견

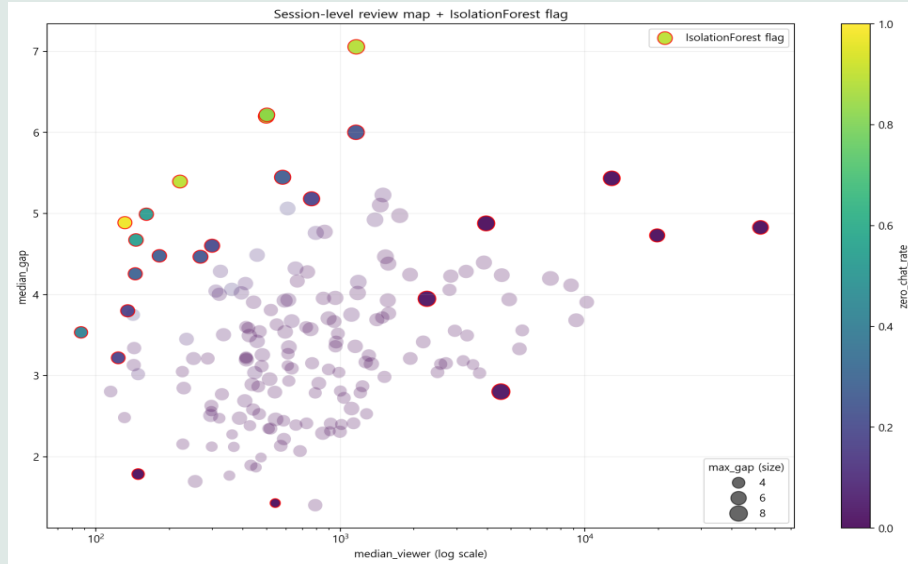
단순히 anomaly detection이기에, 오히려 시청자 수 대비 채팅 수가 월등히 높은 일부 데이터들이 anomaly에 포함되었음 - 이는 우리가 원하는 의심군과 정확히 대조되는 스트리밍도 View-bot 사용 중으로 판별함.

지나치게 많은 View-Bot 의심 Streaming - 전체의 약 14%가 의심 군이라면 지나치게 진보적인 분류기이며, 오히려 분류의 의미가 없음

3.1. Isolation forest 기반

Isolation forest기반 고립 세션 flagging

모델 진행 결과



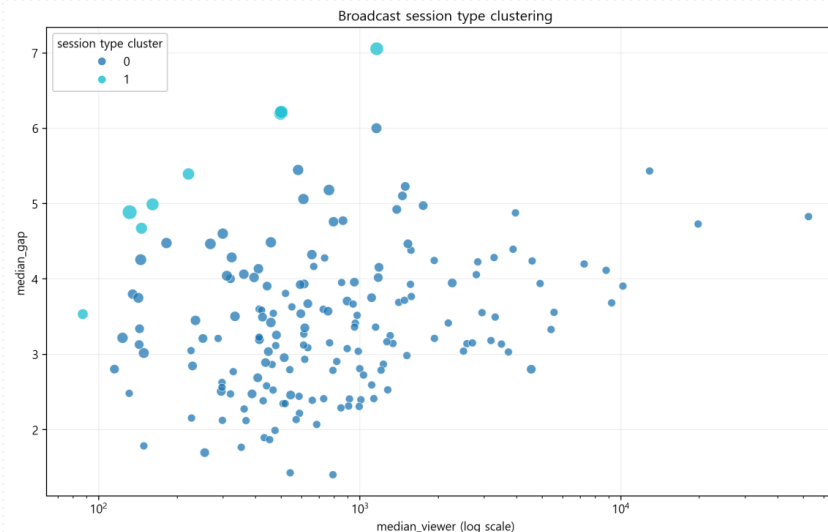
- 결론적으로, Anomaly Detection 만으로는 우리가 원하는 View-Bot Detection을 완수할 수 없음.
가장 major 한 이유는 이는 우리가 원하는 의심군과 정확히 대조되는 스트리밍도 View-bot 사용중으로 판별하기 때문임.

- 오직 View-Bot 의심군만을 탐지하는 모델을 만들어보자!

- 그런데... 무엇이 View-Bot 의심군인지 어떻게 정의하지?

isolation Forest 기반 모델 및 이전의 K-means 클러스터링 결과를 보면, View-Bot steaming은 다양한 Feature에서 Normal Streaming과 차이를 보이고 서로 군집을 이루는 것으로 나타남.

따라서, 이전의 K-means에서 따로 분류되었던 8개를 View-Bot 의심 세션으로 두고 모델 학습을 진행해보자!



- 막간 상식:
What is SHAP?

Mechanism: 각 Feature를 모델에서 포함시켰다가, 제외시키며 기여한 몫을 계산함.

복잡한 모델의 판단 근거를 시각적으로 제시

3.2.1. Decision Tree

Silver Label(K-Means 8개 군집) 기반의 의사결정 규칙 도출 및 분류

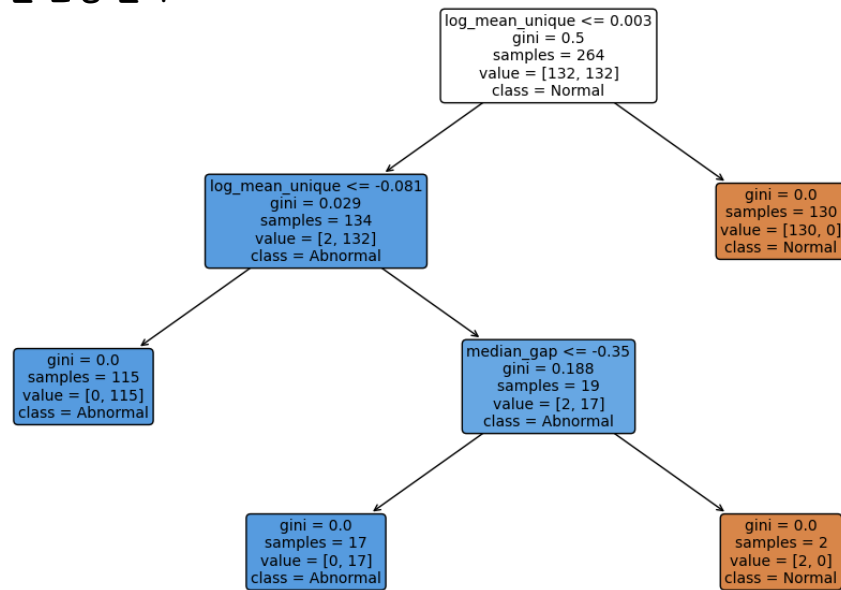
Why Decision Tree?

K-means Cluster가 분류해낸 8개의 Streaming을 정답으로 가정했을 때, 그들은 다른 일반적인 Streaming과 어떤 차이를 내는지 확실한 기준을 가지고 볼 수 있기 때문.

즉 어떤 Feature들이 분류에 영향을 주는지 파악 가능함.

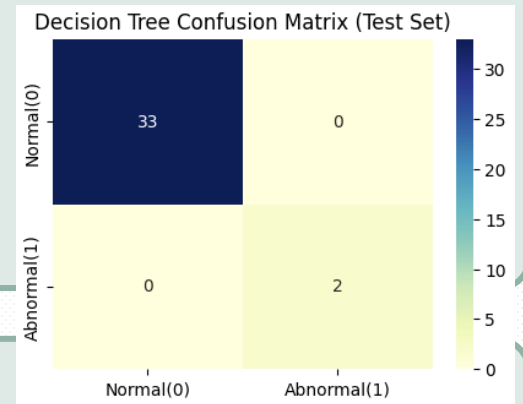
모델 진행 결과

Decision Tree Rules



- Decision Tree의 정확도를 판별하기 위해 Train-Test Split을 8:2, Stratify 옵션을 켜고 진행. Features에는 RobustScaler 사용.

- 사용된 판단 기준은 총 2개 (log_mean_unique x2, median_gap)
log_mean_unique가 생각보다 중요한 Feature일 가능성



- 정확도가 매우 높음 (Confusion Matrix 기준 완벽 분류)
Decision Tree 만으로 매우 의미있는 분류기?

- 몇 가지 문제점

현재 Test Data 내부 View-Bot 분류 데이터가 2개 뿐
- 순전히 운에 의한 Accuracy 100%라 보는 것이 타당

이 문제를 해결하기 위해, 현재 추가 데이터를 수집 중.
추후 데이터가 많아진다면, Decision Tree로는 분류가 힘들 것으로 예상.

- 확률적 접근이 불가능

Decision Tree는 0 or 1 Classification: 이 방송이 어느 정도의 확률로 View-Bot 의심인지 판단이 불가능. 만약 Boundary 근처라면?

3.2.2. Gaussian Naive Bayes

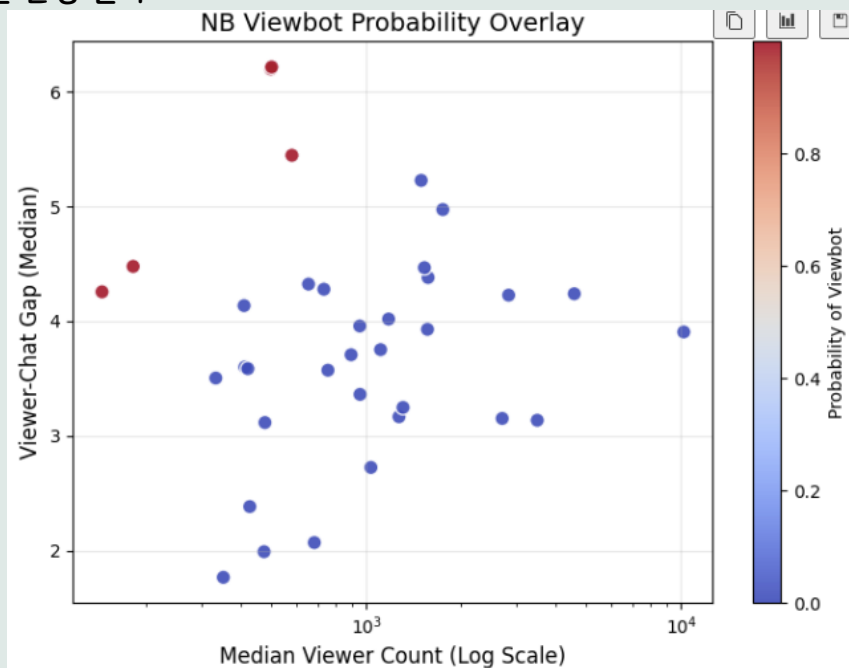
Silver Label(K-Means 8개 군집) 기반의 의사결정 규칙 도출 및 분류

Why Gaussian Naive Bayes?

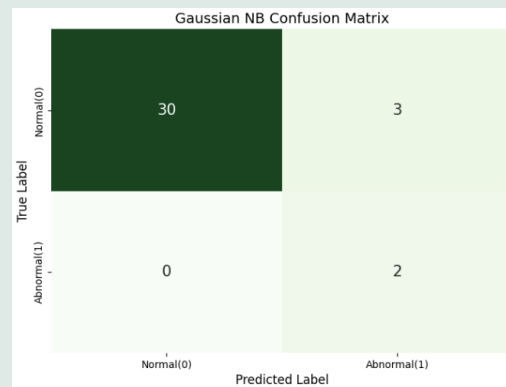
Decision Tree의 확률 접근 불가 문제를 해결하기 위한 고전적 베이스라인 모델

베이지 정리를 기반으로 각 Feature의 데이터 분포를 통해 View-bot일 확률을 연속적으로 도출함

모델 진행 결과



- Train-Test Split을 8:2, Stratify 옵션을 켜고 진행, RobustScaler 사용



Accuracy: 91.4%
Recall: 100% (중요)
Precision: 40%

- 특징

👍 Recall 100%: 긍정적인 부분은 단 하나의 View-Bot 의심군도 놓치지 않고 100% 확률로 적발해 냈다는 점.
이상탐지 목적상 실제 View-Bot을 모두 숙아낸 것은 Baseline 모델로서 좋은 성과다.

⚠️ Overconfidence와 Precision 40%: 하지만 변수 간의 복잡한 상관관계를 무시하는 독립성 가정(Naïve)이 있음.
그 결과, 조금만 이상해도 과잉 확신하는 경향성, 거의 비율상으로는 이전의 Isolation Forest와 비슷한 14%.

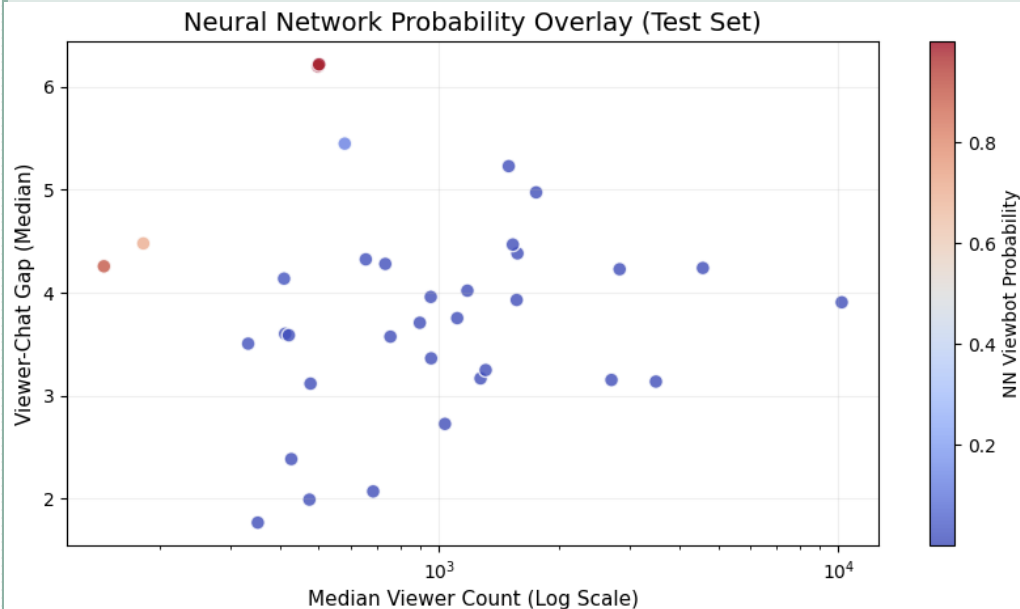
3.3. Classification Neural Network

Silver Label(K-Means 8개 군집) 기반의 의사결정 규칙 도출 및 분류

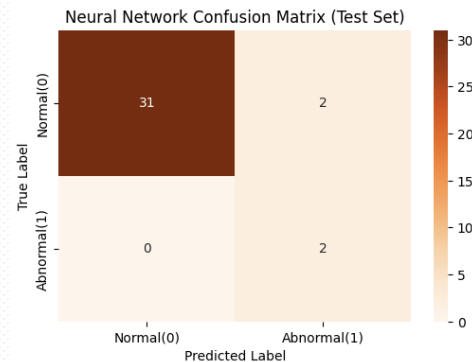
Why Neural Network?

비선형성(Non-linearity) 학습: 시청자 수, 채팅 빈도, median_gap 등 다양한 변수들이 서로 어떻게 얹혀서 View-Bot의 특징을 만들어내는지(복잡한 상관관계)를 은닉층(Hidden Layer)을 통해 유연하고 정교하게 학습함.

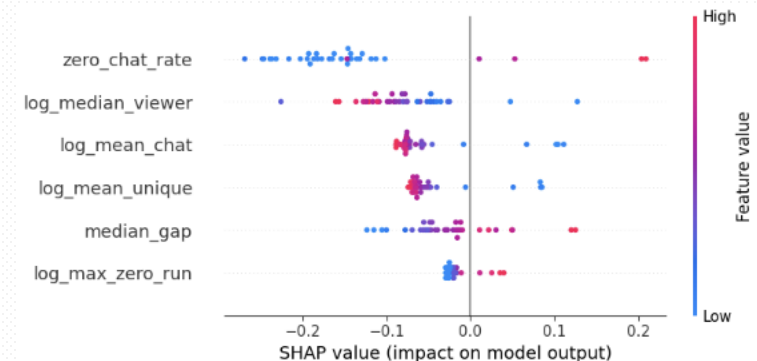
모델 진행 결과



- Train-Test Split을 8:2, Stratify 옵션을 켜고 진행, RobustScaler 사용
- Hidden Layer: (16, 8,), Activation Func: ReLu, alpha: .0001, iter = 1,000, optimizer: adam



Accuracy: 94.3%
Recall: 100% (중요)
Precision: 50%



- 특징

개선된 정밀도: View-Bot 적발 능력은 유지하면서, False Positive 분류율을 줄이는데 성공함.

분류율의 차이: 극단적으로 분류하지 않고, 경계면에 위치한 데이터들에 대해 조금 더 View-Bot 의심 확률을 높게 잡음

- 한계

Baysian Model과 비교해서 거의 성능 향상이 없음.
NN에 학습하기에는 데이터 수가 너무 적음

3.4. 정리

진행했던 4개의 모델 결과 비교 및 개선점 파악

- isolationForest

: View-Bot이 다양한 부분에서 일반적인 Streaming과 차이점을 보인다는 것을 실증적으로 증명함, 한편 원하지 않는 방향의 anomal까지 Detect되고 지나치게 많은 방송을 의심함

- DecisionTree

: 완벽하게 분류함, 하지만 데이터 수가 부족하기 때문일 확률이 큼. DecisionTree에 따르면 주요한 판단 기준은 log_mean_unique x2, median_gap

- Naïve Bayes

: 정확도 91%, Recall 100%, Precision 40%
기본 모델인만큼 정확도가 낮음.

- Neural Network

: 정확도 94%, Recall 100%, Precision 50%
Naïve Bayes 모델보다는 발전했지만, Neural Network가 이론적으로 가지는 수학적 결정력을 생각하면 아쉬움.
SHAP에 따르면 주요한 판단 기준은 zero_chat_rate, median_gap, log_max_zero_run

- 주목할만한 point

가장 신경쓰이는 부분: DT와 NN 모델에서, 판단 기준이 서로 상이함.
이는 View-Bot 방송이 여러 차원에서 차이를 보인다는 것을 실증하지만, 동시에 다음과 같은 한계점을 증명함

1. K-means 8개 군집 자체가 불완전한 가설일 수 있음. - 모델들은 각자 다른 방식으로 억지로 View-Bot의 특징을 과적합했을 수 있음.
2. 데이터의 수가 매우 부족함 - 근본적으로 View-Bot 데이터의 수가 8개로 매우 적기 때문에, 다양한 기준으로도 무리없이 분류되었을 것임.
3. 단일 모델(Global Model)의 한계 - '정상 vs View-Bot'의 단순 이진 분류로는 방송 데이터 내부의 다양한 세부 패턴(이질성)을 온전히 담아낼 수 없음

- 필요 개선점

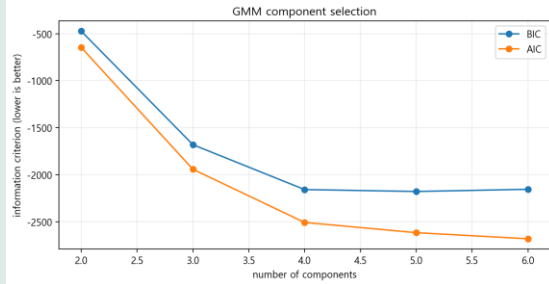
1. 데이터 수의 추가: 현재 데이터는 약 2주 전까지의 데이터로, 지속적으로 데이터를 모으고 있으며 기말까지 약 현재 데이터의 2배를 추가로 모을 것으로 예상
2. 새로운 모델의 필요성: 많은 정상 데이터들을 학습시킨 뒤, 정상 범주를 넘어서는 것을 View-Bot으로 색출하는 형태의 AutoEncoder 모델이 필요함
3. Clustering을 이진 분류로 하지 않고, 전체 구조를 다시 파악해보기?

PART. 04

향후 프로젝트 계획

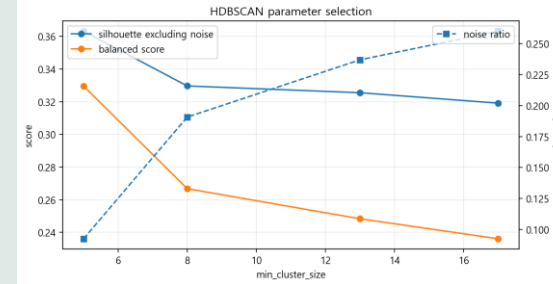
4.1 From Kmeans-based Global Model to Cluster-aware Modeling

GMM Clustering & HDBSCAN



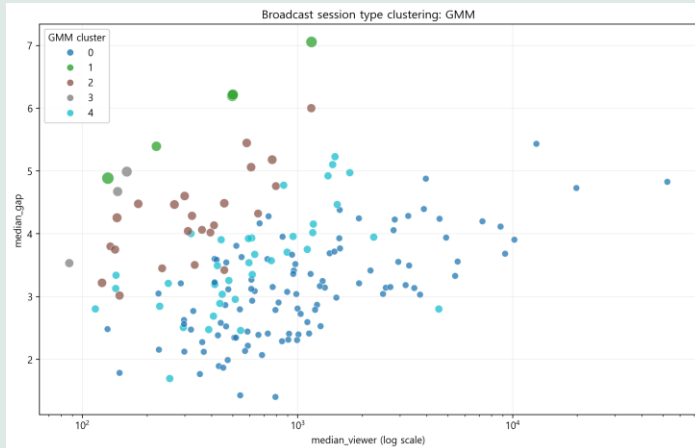
n	bic	aic	silhouette
4	-2158.18	-2508.2	0.271
5	-2178.72	-2617.03	0.301
6	-2155.28	-2681.88	0.229

BIC minimum: 5 components
AIC minimum: 6 components
→ GMM suggests hidden sub-structures

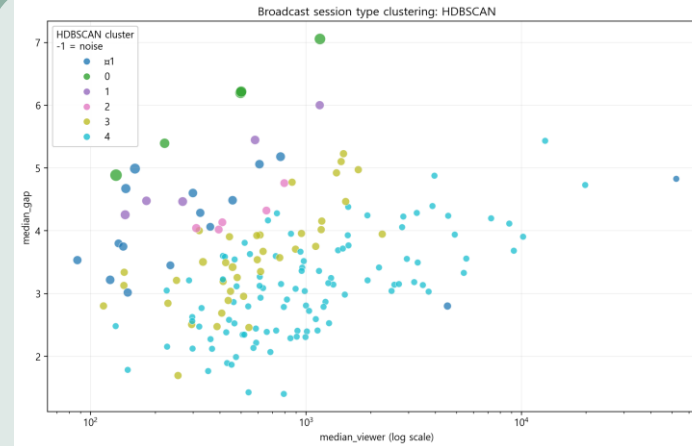


min_size	n_clusters	n_noise	coverage	score
5	5	16	0.908	0.330
8	2	33	0.809	0.267

min_cluster_size = 5 gives the best balance:
5 clusters, 16 noise sessions,
coverage = 90.8%, score = 0.330



Active component: n=105, viewer=907, chat/min=73.4
Mismatch-like component: n=5, viewer=498, chat/min=0.21, zero-chat=88.8%



HDBSCAN separates stable clusters and noise

Cluster 4: active normal
n=104, viewer=902, chat/min=70.2

Noise: 16 manual-review candidates

Cluster 0: mismatch candidate
n=5, viewer=498, chat/min=0.21, zero-chat=88.8%, max zero-run=26

유의사항: 6개의 요소를 cluster feature로 넣었고 2D plot을 했기에 cluster간에 혼합되어 보일 수 있음

결론: Kmeans는 큰 구분에는 유용했지만, GMM과 HDBSCAN은 세션 내부의 **이질성**을 보여준다.



기말 모델은 **cluster-aware modeling**으로 확장

4.2.1. 기말 발표까지 개선할 요소들

기말에는 조금 더 좋은 모델을 만들어 의미있는 결과를 낼 수 있길...

- 절대적 데이터 수의 추가:

실질적으로 가장 큰 모델 성능 향상을 가져올 핵심 요소. 현재 173개 세션, 단 8개의 View-Bot 의심군이라는 극소수 데이터가 가진 통계적 불안정성 및 Overfitting 문제를 해결하기 위해, 대규모 추가 크롤링을 진행하여 모델의 일반화 성능을 확보

예상되는 데이터 수는 약 450~500개 세션

- 거시적 이분법에서 군집 인지형 모델(Cluster-aware Modeling)로 전체 방송을 '정상 vs 비정상'으로 묶어버리는 획일적인 단일 분류기 폐기. HDBSCAN 등으로 파악된 방송 유형(예: 소통 활발형, 게임 집중형 등)을 먼저 분류한 뒤, 각 군집별 특성에 맞는 독립적인 이상 탐지 기준을 적용하는 고도화된 파이프라인 구축.

- Ground-Truth 문제 해결

현재 데이터는 결국 자의적인 판단이 가미되었고, 실질적으로 View-Bot인지는 알 수 없는 상황임. **가능하다면** 실제로 View-Bot 관련 Dataset을 찾아보거나, 최근 View-Bot에 의해 정지된 스트리밍의 데이터를 수집 (하지만 실질적으로는 쉽지 않을 것으로 예상)

- 더 고도화된 모델 사용

1. 기본적인 모델로는 SVM, LOF을 추가.
2. 데이터 labeling이 명확하지 않은 상황이기 때문에, 비지도(또는 반지도) 학습법: 그 중에서도 PU-Learning와 AutoEncoder는 역시 타당하고 반드시 필요한 접근법.
3. 의사결정나무가 성능이 좋았기 때문에, 추가로 XGBoost, LightGBM

- 본질적으로 원래 데이터는 시계열 데이터

시계열 데이터를 가공한 현재 데이터로 충분히 유의미한 결과가 나오지 않는다면, 시계열 데이터 자체를 바탕으로 모델을 학습해야 함.

1. 기본적인 모델로는 Sliding Window, tsfresh
2. 1D-CNN, LSTM / GRU 등의 신경망 모델
3. LSTM - AutoEncoder (만약 가능하다면)

- 실시간 Tracking 문제

현재 모델은 방송 도중 판단하는 것이 아니라 데이터를 추출한 이후 모델을 바탕으로 판단함. 방송 도중 이를 판단할수 있도록 파이프라인을 짜면 실질적인 도움이 될 것임.

4.2.2. 구체적인 실험/튜닝 계획

기말에는 조금 더 좋은 모델을 만들어 의미있는 결과를 낼 수 있길...

- 하이퍼파라미터 최적화 (~5/26)

GridSearch는 기본적으로 진행. NN의 경우 아직 GridSearch를 진행하지 않았기 때문에, 많은 성능 향상이 있을 것으로 예상.

더 나아가, Optuna 등의 베이지안 최적화(Bayesian Optimization) 프레임워크를 도입.

이를 NN에 적용, 그리고 추가로 lightGBM의 학습률이나 num_leaves, 그리고 AutoEncoder에서도 사용할 예정

- Cross-Validation 도입 (~5/26)

현재의 단일 Train-Test Split(8:2) 구조는 우연에 의해 성능이 높게 측정되었을 가능성이 있음. Stratified K-Fold Cross Validation (K=5 또는 10)을 적용하여, 어떤 데이터가 Test 셋으로 빠지더라도 성능이 흔들리지 않는지 모델의 분산과 강건성(Robustness) 확인

- 다중 Classifier 결과 적용 (~5/15)

기존 K-Means Clustering에서 K=2일 때의 결과를 바탕으로 Baseline 모델을 학습시켰는데, K=2보다 더 다양하게 Clustering되는 결과를 확인했기 때문에 기존 모델들을 수정해야 함.

- 특화된 평가지표(Metric) 전환 (~5/15)

View-Bot 데이터가 극소수인 환경에서는 Accuracy(정확도)가 의미를 상실함. Recall이 훨씬 중요함.

향후 실험 평가는 Precision과 Recall의 가중조화평균인 F_beta-Score와, 불균형 데이터 평가에 가장 강력한 PR-AUC (Precision-Recall Area Under Curve) 지표를 최우선 기준으로 삼아 모델을 최적화

- Ablation Study을 통한 Feature 검증 (~5/28)

SHAP 분석으로 파악한 핵심 변수(zero_chat_rate, median_gap 등)들을 기말 모델에서 하나씩 의도적으로 제거해가며 성능 하락 폭을 측정.

이를 통해 해당 변수들이 View-Bot 예측에 실제로 얼마나 지배적인 기여를 하는지 정량적으로 교차 검증하고, 불필요한 노이즈 변수를 제거하여 모델의 신뢰도를 확보할 예정.

4.2.3. 기말 프로젝트 로드맵

예상 수행 일정

1

5.15: 기본 모델 수정

새로운 Clustering 결과는 이진 분류가 부적절함을 실증함. 기존 Baseline Model들을 다중 분류기로 업데이트

3

6월 첫째 주: 데이터 수집 마감

5

6월 둘째 주: 실시간 탐지 파이프라인

실시간 파이프라인을 구축하여, 우리의 모델이 상식과 부합하는지 확인. 랜덤한 스트리밍이 뷰봇으로 의심될 때, 모델은 이를 어떻게 판단하는지 검증

5월 마지막 주: 새로운 모델의 추가

Baseline 모델 대비 성능 향상이 어느 정도 이루어지는지 확인하기,

2

6월 첫째 주: 심화 모델 학습

시계열 데이터 기반 모델 학습.

4

기말 발표 준비

마지막 발표, 수행한 모든 것을 보여주기!

6

THANK YOU